CASE STUDY 23

Student Timetable Conflict Detection

1. PROBLEM STATEMENT

Represent course schedules using tuples, detect overlapping periods, identify room clashes, and validate a proposed
timetable.

2. OBJECTIVES

• Store course details using tuples.

• Detect time overlaps for a student or room.

• Report timetable conflicts.

3. ASSUMPTIONS

• Each tuple contains course name, day, start hour, end hour, and room.

• Time is represented using 24-hour integer values.

• Two sessions overlap when they are on the same day and each starts before the other ends.

• A room cannot host two courses at overlapping times.

4. ALGORITHM

1. Store timetable entries as tuples containing course, day, start, end and room.

2. Compare each pair of timetable entries using nested loops.

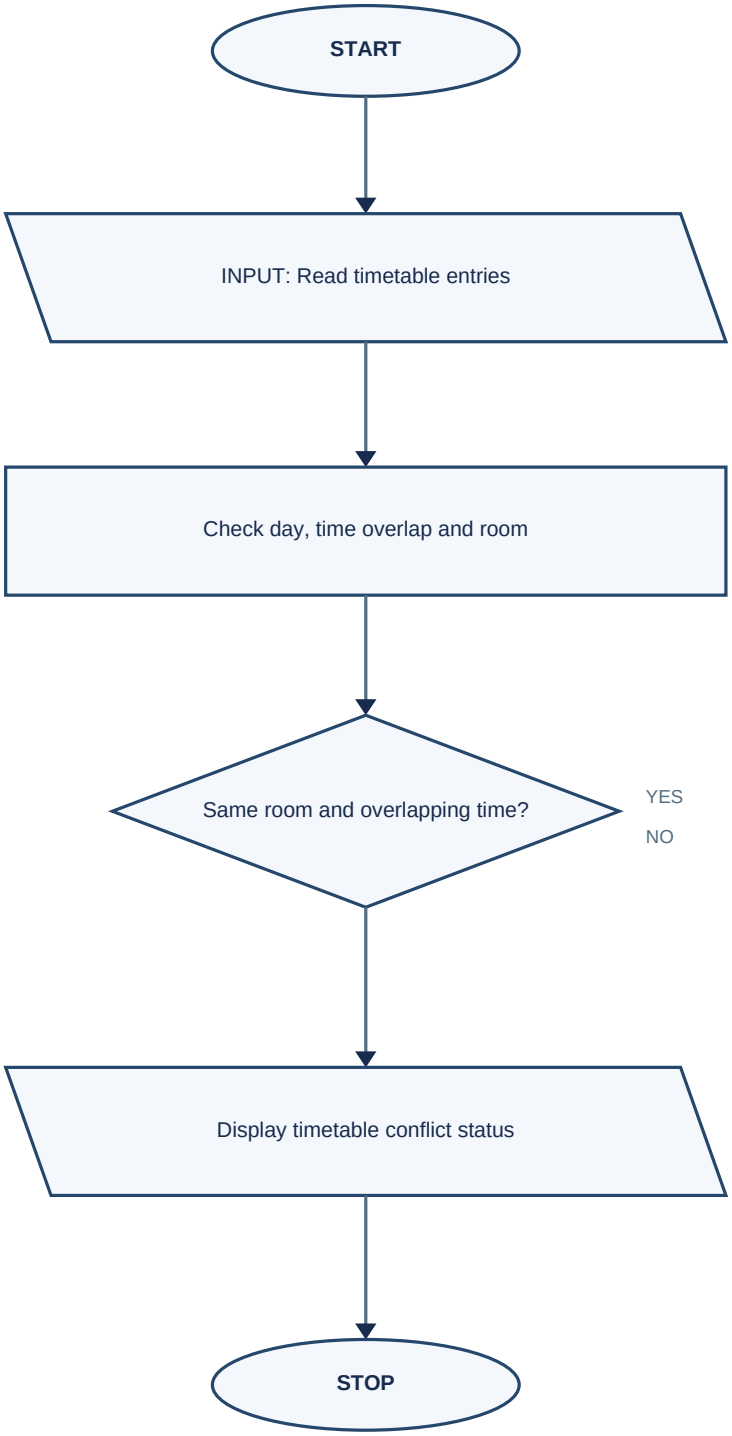
3. Check whether both entries are on the same day and their time intervals overlap.

4. If the room is also the same, record and display a room conflict.

5. Display whether any room conflict was found.

5. FLOWCHART

Oval: START/STOP | Parallelogram: input/output | Rectangle: process/calculation | Diamond: decision



6. PSEUDOCODE

```
BEGIN
INPUT timetable entries as tuples
conflict_found = false
FOR each pair of timetable entries
CHECK whether day is the same
CHECK whether time intervals overlap
CHECK whether room is the same
IF all three conditions are true
DISPLAY room conflict
conflict_found = true
IF no conflict: DISPLAY no room conflicts
ELSE: DISPLAY timetable needs correction
END
```

7. PYTHON SOURCE CODE

```
# Each tuple: (course, day, start, end, room)
timetable = [
("Python", "Monday", 9, 11, "Lab-1"),
("Maths", "Monday", 10, 12, "Room-2"),
("English", "Monday", 10, 11, "Lab-1"),
("Physics", "Tuesday", 9, 10, "Lab-1")
]
conflict_found = False
for i in range(len(timetable)):
for j in range(i + 1, len(timetable)):
a = timetable[i]
b = timetable[j]
same_day = a[1] == b[1]
overlap = a[2] < b[3] and b[2] < a[3]
same_room = a[4] == b[4]
if same_day and overlap and same_room:
print("ROOM CONFLICT:", a[0], "and", b[0])
print("Day:", a[1], "Room:", a[4])
conflict_found = True
if not conflict_found:
print("No room conflicts found")
else:
print("Timetable needs correction")
```

8. TEST CASES & EXPECTED RESULTS

TEST INPUT / CONDITION REASONING EXPECTED RESULT

Python Monday 9–11 Lab-1; English Same day, overlapping time and room Conflict detected

Monday 10–11 Lab-1

Python Monday 9–11 Lab-1; Maths Different rooms No room conflict

Monday 10–12 Room-2

Physics Tuesday 9–10 Lab-1 Different day from Monday entries No conflict with Monday entries

9. SAMPLE OUTPUT

```
ROOM CONFLICT: Python and English
Day: Monday Room: Lab-1
Timetable needs correction
```

10. CONCLUSION

The program uses tuples and nested loops to find overlapping room bookings and helps prevent timetable

allocation

errors.

SYLLABUS CONCEPTS Variables • Data Types • Operators • Conditions • Loops • Lists • Nested Lists • Tuples • String Operations